



資料結構

Data Structure

Lab 10

姓名：王顥程
學號：114310215

Lab10 (題目編號)**Exercise: Complement heapify function****Code**

```
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
using namespace std;

vector<int> readFromFile(const string& filename) {
    vector<int> arr;
    ifstream file(filename); // 打開檔案輸入流

    // 檢查檔案是否成功打開, 若失敗則印出錯誤訊息並返回空向量
    if (!file) {
        cerr << "Error opening file: " << filename << endl;
        return arr;
    }

    string line;
    // 使用 getline 逐行讀取檔案內容
    while (getline(file, line)) {
        stringstream ss(line); // 將讀取到的字串轉為字串流, 方便分割
        string value;
        // 以逗號 ',' 為分隔符號, 切出每一個數字字串
        while (getline(ss, value, ',')) {
            try {
                // 將字串轉為整數 (stoi) 並推入 vector 中
                arr.push_back(stoi(value));
            }
            catch (exception& e) {
                // 如果轉換失敗 (例如遇到非數字字元), 捕捉異常並提示
                cerr << "Invalid number format in file: " << value << endl;
            }
        }
    }

    file.close(); // 關閉檔案流
    return arr;
}

// =====
// ===== Max Heap 類別 (最大堆積) =====
```

```
// =====
class MaxHeap {
public:
    vector<int> heap; // 儲存 Max Heap 元素的動態陣列

    void buildMaxHeap(vector<int>& arr) {
        heap = arr; // 先將原始數據複製到 heap 陣列中

        // 從最後一個「非葉子節點」開始, 路由下往上、由右往左依序進行 heapify
        // 公式: 最後一個非葉子節點的索引為 (大小 / 2) - 1
        for (int i = (heap.size() / 2) - 1; i >= 0; i--) {
            heapify(i);
        }
    }

    void heapify(int i) {
        int largest = i; // 先假設當前的父節點 i 是自己和子節點中最大的
        int left = 2 * i + 1; // 根據二元樹公式計算左子節點的索引
        int right = 2 * i + 2; // 根據二元樹公式計算右子節點的索引

        // 1. 檢查左子節點是否存在 (未越界), 且其值是否大於目前設定的最大值
        if (left < heap.size() && heap[left] > heap[largest]) {
            largest = left; // 若成立, 將最大值索引更新為左子節點
        }

        // 2. 檢查右子節點是否存在 (未越界), 且其值是否大於目前設定的最大值
        if (right < heap.size() && heap[right] > heap[largest]) {
            largest = right; // 若成立, 將最大值索引更新為右子節點
        }

        // 3. 如果最大值的索引不再是原本的父節點 i, 代表子節點比較大, 必須交換
        if (largest != i) {
            swap(heap[i], heap[largest]); // 將父節點與較大的子節點數值對調
            heapify(largest);
        }
    }

    /**
     * @brief 以廣度優先 (BFS) 順列印出目前的 Heap 陣列內容
     */
    void printHeap() {
        for (int val : heap) {
            cout << val << " ";
        }
    }
};
```

```

    }
    cout << endl;
}
};

// =====
// ===== Min Heap 類別 (最小堆積) =====
// =====

class MinHeap {
public:
    vector<int> heap; // 儲存 Min Heap 元素的動態陣列

    void buildMinHeap(vector<int>& arr) {
        heap = arr; // 先將原始數據複製到 heap 陣列中

        // 同樣從最後一個「非葉子節點」開始由下往上做 heapify
        for (int i = (heap.size() / 2) - 1; i >= 0; i--) {
            heapify(i);
        }
    }

    void heapify(int i) {
        int smallest = i;    // 先假設當前的父節點 i 是自己和子節點中最小的
        int left = 2 * i + 1; // 左子節點索引
        int right = 2 * i + 2; // 右子節點索引

        // 1. 檢查左子節點是否存在, 且其值是否比目前設定的最小值還要小
        if (left < heap.size() && heap[left] < heap[smallest]) {
            smallest = left;    // 若成立, 將最小值索引更新為左子節點
        }

        // 2. 檢查右子節點是否存在, 且其值是否比目前設定的最小值還要小
        if (right < heap.size() && heap[right] < heap[smallest]) {
            smallest = right;    // 若成立, 將最小值索引更新為右子節點
        }

        // 3. 如果最小值的索引不再是原本的父節點 i, 代表子節點比較小, 必須交換
        if (smallest != i) {
            swap(heap[i], heap[smallest]); // 將父節點與較小的子節點數值對調
            heapify(smallest);
        }
    }
}

```

```

void printHeap() {
    for (int val : heap) {
        cout << val << " ";
    }
    cout << endl;
}
};

// =====
// ===== 主程式進入點 =====
// =====

int main() {
    // 設定要讀取的數據檔案路徑 (請確保路徑中的反斜線使用雙反斜線 \\ 進行跳脫)
    string filename = "C:\\Users\\CNC\\Desktop\\input1.txt";

    // 呼叫函式讀取檔案並取得陣列
    vector<int> arr = readFromFile(filename);

    // 檢查 vector 是否為空, 若為空代表沒讀到檔案或格式全錯, 終止程式
    if (arr.empty()) {
        cerr << "No valid data found in file." << endl;
        return -1;
    }

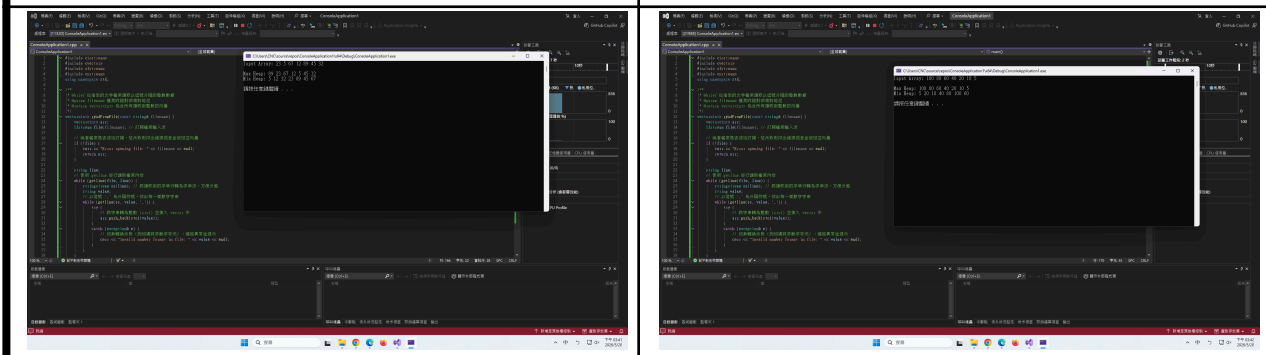
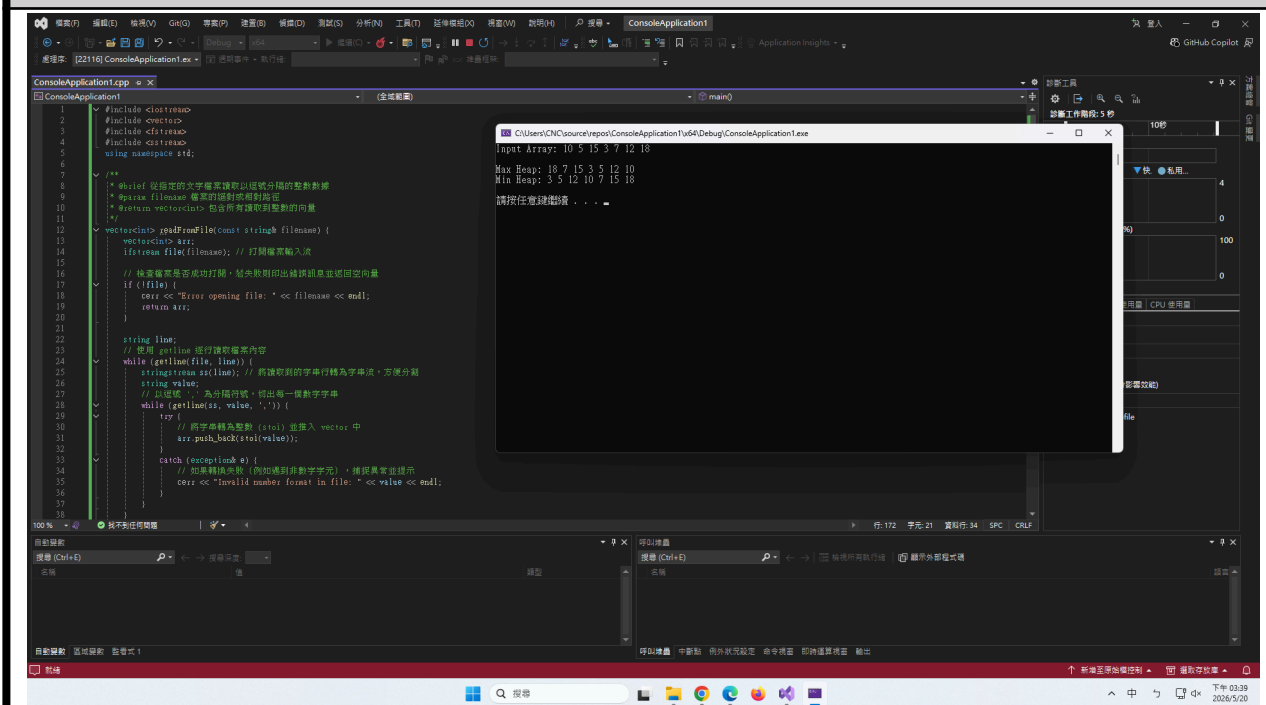
    // 1. 輸出從檔案中讀取到的原始未排序陣列
    cout << "Input Array: ";
    for (int val : arr) {
        cout << val << " ";
    }
    cout << "\n\n";

    // 2. 實例化 MaxHeap 物件, 建構並輸出 Max Heap 結果
    MaxHeap maxHeap;
    maxHeap.buildMaxHeap(arr);
    cout << "Max Heap: ";
    maxHeap.printHeap();

    // 3. 實例化 MinHeap 物件, 建構並輸出 Min Heap 結果
    MinHeap minHeap;
    minHeap.buildMinHeap(arr);
    cout << "Min Heap: ";
    minHeap.printHeap();
    cout << endl;
}

```

```
system("pause");  
return 0;  
}
```

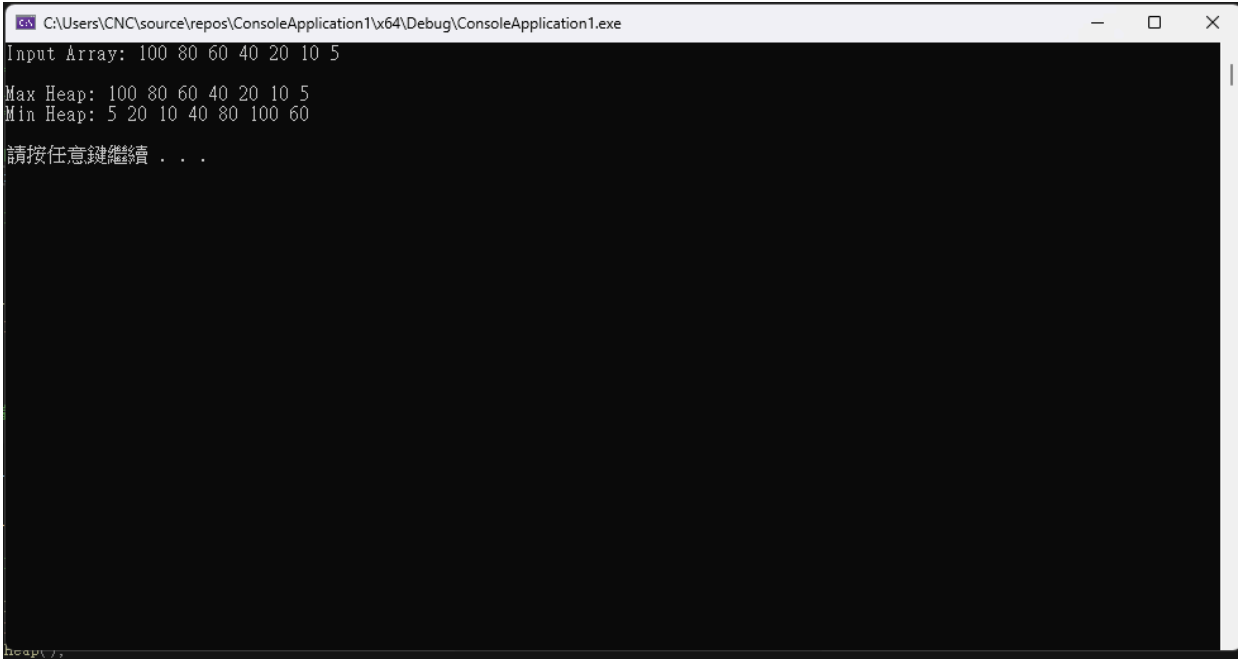


```
C:\Users\CNC\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe
Input Array: 10 5 15 3 7 12 18
Max Heap: 18 7 15 3 5 12 10
Min Heap: 3 5 12 10 7 15 18
請按任意鍵繼續 . . .
```

Lab01-Q1. (題目編號)

Code

```
C:\Users\CNC\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe
Input Array: 23 5 67 12 89 45 32
Max Heap: 89 23 67 12 5 45 32
Min Heap: 5 12 32 23 89 45 67
請按任意鍵繼續 . . .
```


Result
Discussion

--

Lab01-Q#. (題目編號)
Briefly describe the problem. (題目簡述)
Code

Paste your code here. Please use Pygments to highlight your code. (請使用Pygments轉換程式碼格式後再貼上)		Paste your code here. Please use Pygments to highlight your code. (請使用Pygments轉換程式碼格式後再貼上)	
result			
(a)	(b)	(c)	
	Show your result in tables or screenshots. (請使用表格或是截圖呈現結果。)		
Discussion			
Discuss and conclude your results, or answer questions. (實驗討論、結論或回答問題。)			